Informe Técnico: API REST con Django y Pruebas en Postman

Proyecto: uva_django

Autor: Juan Camilo Arenas Gutierres

Fecha: 04 de noviembre de 2026

Entorno: Django (servidor de desarrollo en http://127.0.0.1:8000), Postman

1. Introducción

Este informe documenta el desarrollo y la validación de una API REST construida con Django que administra el recurso **uvas**. Cada registro tiene los campos `id`, `peso` (decimal), `tamano` (entero), `forma` (texto) y `textura` (texto). Se probaron las cinco operaciones del CRUD con Postman y se verificó en la terminal del servidor que cada petición se registrara correctamente.

Operación	Método HTTP	Endpoint	Código esperado
Listar todos	GET	/api/uvas/	200 OK
Consultar uno	GET	/api/uvas/{id}/	200 OK
Crear	POST	/api/uvas/	201 Created
Editar	PUT / PATCH	/api/uvas/{id}/	200 OK
Eliminar	DELETE	/api/uvas/{id}/	204 No Content

2. Panel de Administración de Django

Django incluye un panel administrativo (`/admin/`) que permite gestionar los modelos registrados. Allí se gestionó el primer registro de prueba (`id: 1`), que luego se consultó y modificó desde la API. Esto confirma que el modelo está definido, registrado en `admin.py` y migrado a la base de datos.

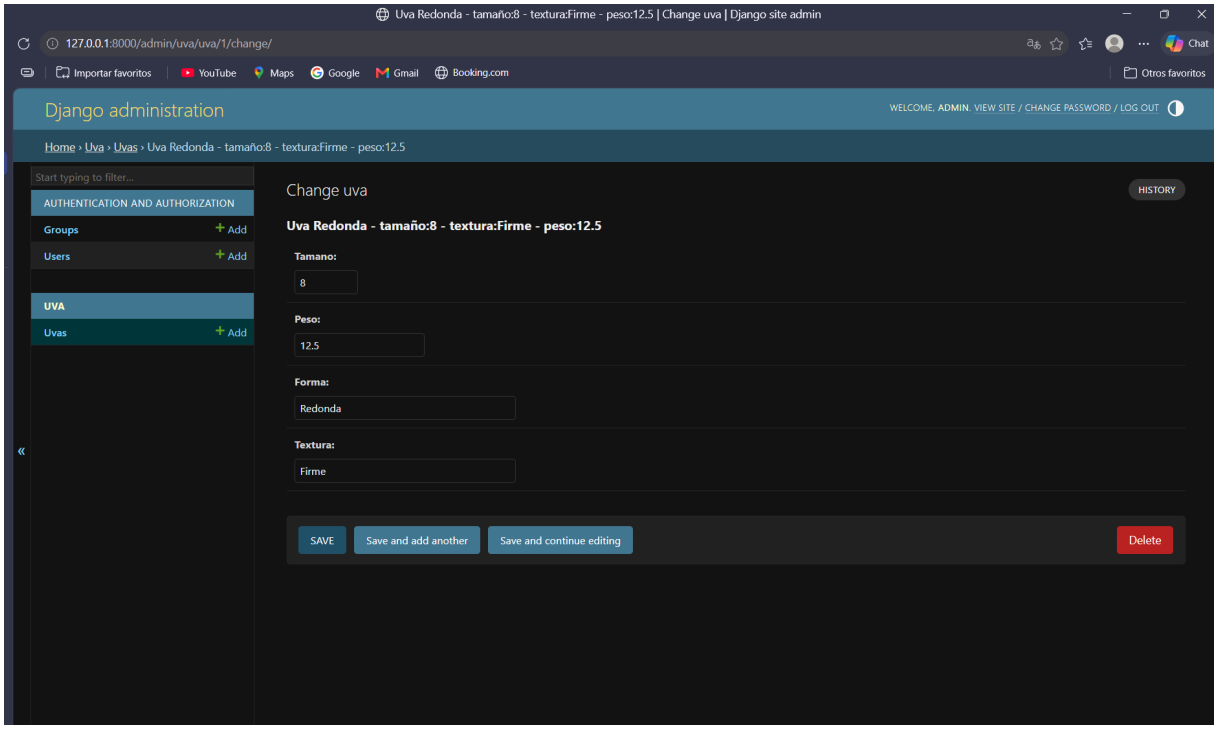


Figura 1. Formulario de edición del registro de uva (id 1) en el panel de administración de Django. Se observa la textura "Firme", valor actualizado desde la API con PUT.

3. Pruebas de la API REST en Postman

3.1 Petición GET: listar todos los registros

Método y URL: GET http://127.0.0.1:8000/api/uvas/ | **Resultado:** 200 OK (18 ms, 397 B)

La API devuelve una lista JSON con todos los registros almacenados. En este momento contiene un único elemento.

```
[
    {"id": 1, "peso": 12.5, "tamano": 8,
     "forma": "Redonda", "textura": "blanda"}
]
```

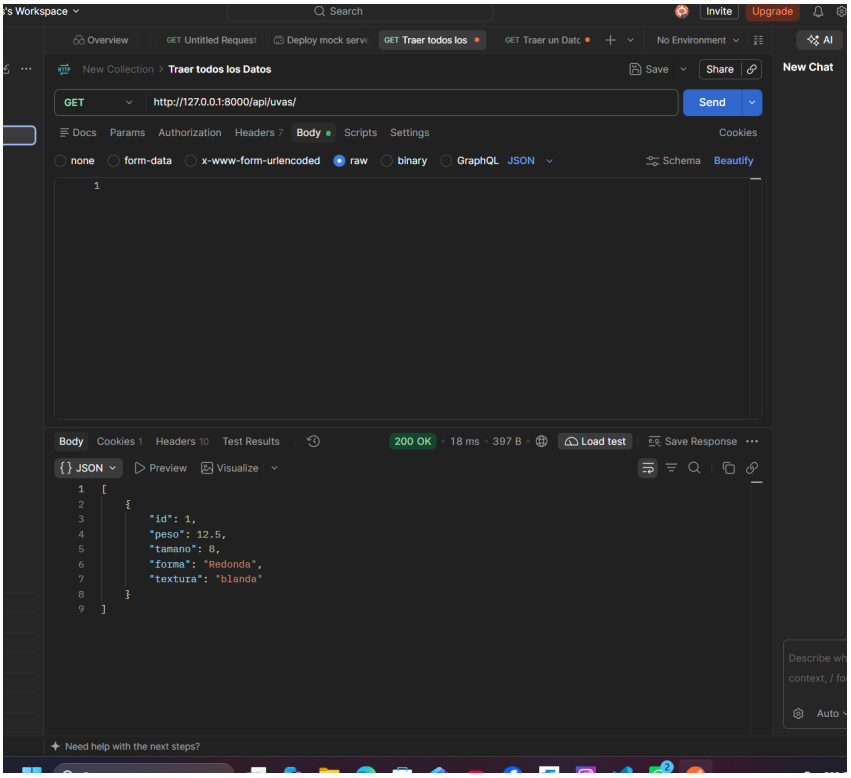


Figura 2. Respuesta del endpoint de listado.

3.2 Petición GET por ID (/api/uvas/1/)

Método y URL: GET http://127.0.0.1:8000/api/uvas/1/ | **Resultado:** 200 OK (7 ms, 409 B)

El endpoint de detalle devuelve un único objeto JSON (no una lista) con el registro cuyo identificador coincide con el de la URL.

```
{"id": 1, "peso": 12.5, "tamano": 8,
 "forma": "Redonda", "textura": "blanda"}
```

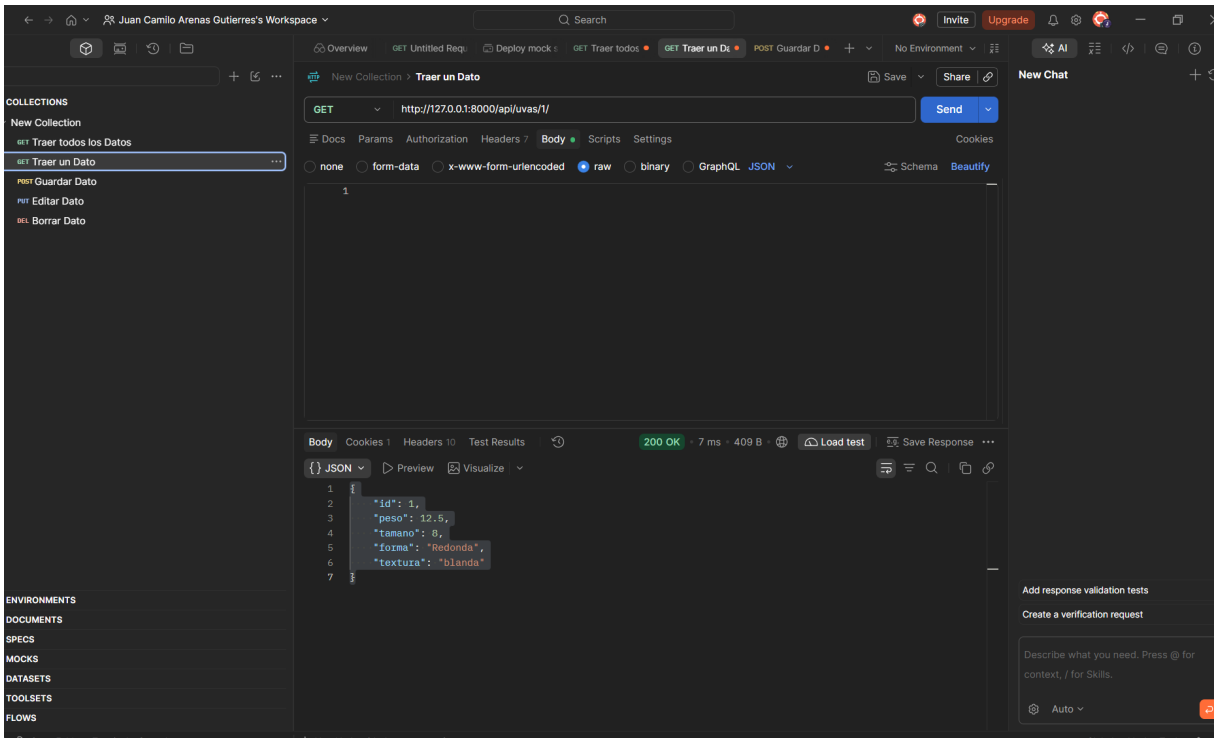


Figura 3. Consulta de un registro específico.

3.3 Petición POST: crear un nuevo registro

Método y URL: POST http://127.0.0.1:8000/api/uvas/ | **Resultado:** 201 Created (31 ms, 403 B)

Se envió un cuerpo JSON (raw → JSON) con los datos del nuevo registro. El servidor lo almacenó, le asignó automáticamente el `id: 2` y devolvió el objeto creado, lo que confirma la persistencia.

```
Cuerpo enviado:
{"peso": 12.5, "tamano": 8, "forma": "Redonda", "textura": "crujiente"}

Respuesta:
{"id": 2, "peso": 12.5, "tamano": 8, "forma": "Redonda", "textura": "crujiente"}
```

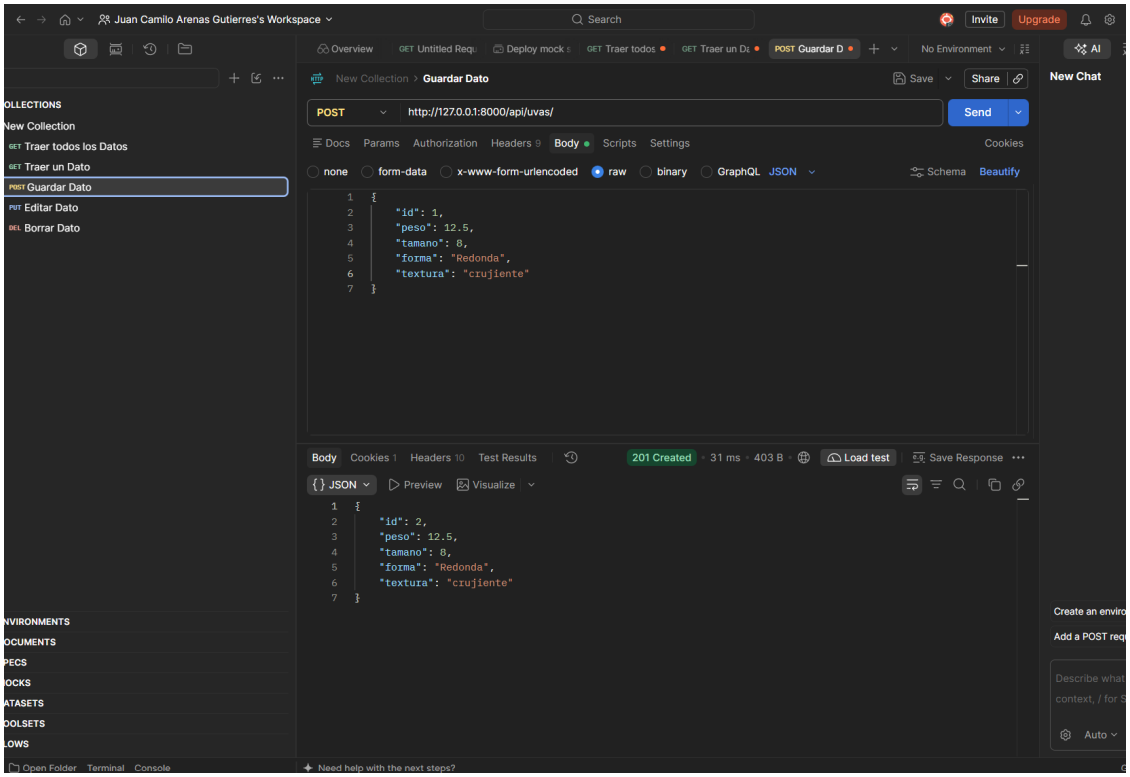


Figura 4. Creación de un registro con respuesta 201 Created.

3.4 Petición PUT: editar un registro

Método y URL: PUT http://127.0.0.1:8000/api/uvas/1/ | **Resultado:** 200 OK (28 ms, 408 B)

Se actualizó el registro con `id: 1`, cambiando el campo `textura` de "blanda" a "Firme". PUT reemplaza el recurso completo, por lo que el cuerpo incluye todos los campos. La respuesta devuelve el registro ya modificado.

```
{"id": 1, "peso": 12.5, "tamano": 8,
 "forma": "Redonda", "textura": "Firme"}
```

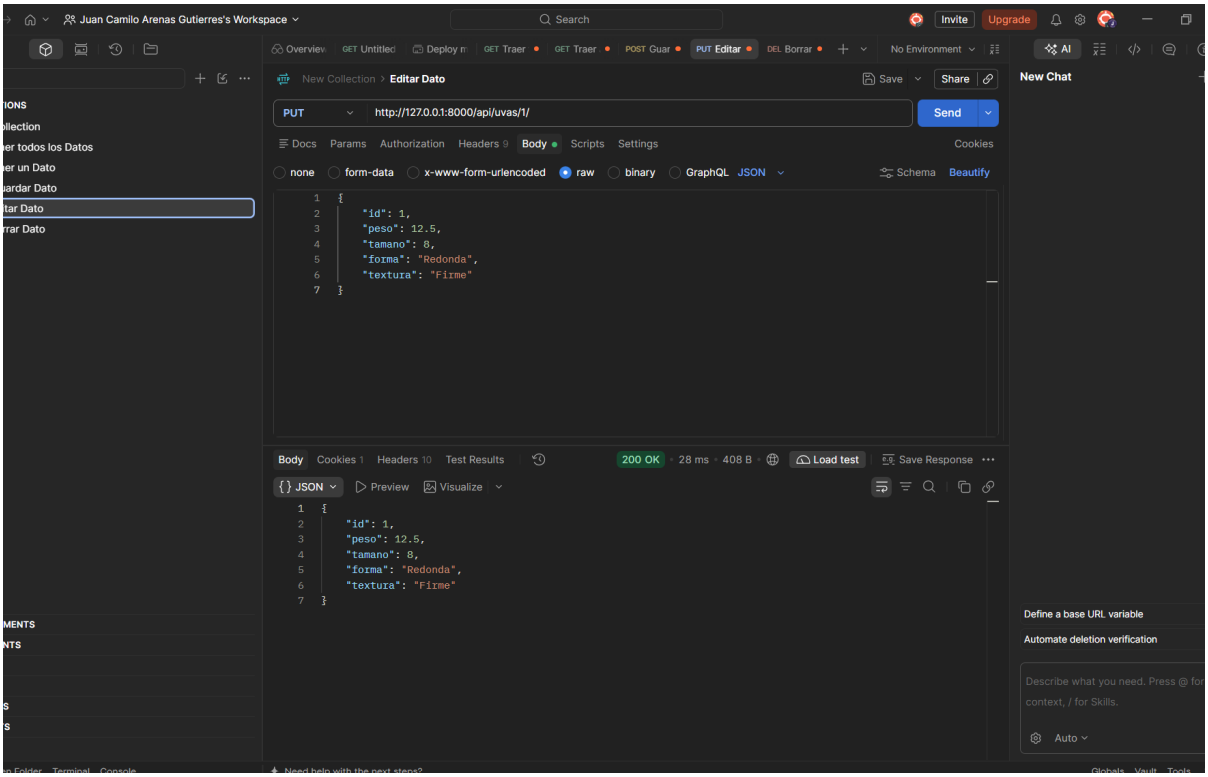


Figura 5. Actualización del registro con respuesta 200 OK.

3.5 Petición DELETE: eliminar un registro

Método y URL: DELETE http://127.0.0.1:8000/api/uvas/2/ | **Resultado:** 204 No Content (27 ms)

Se eliminó el registro creado en la prueba POST (`id: 2`). El código 204 indica que la operación fue exitosa y que el servidor no devuelve contenido, por eso el cuerpo de la respuesta aparece vacío.

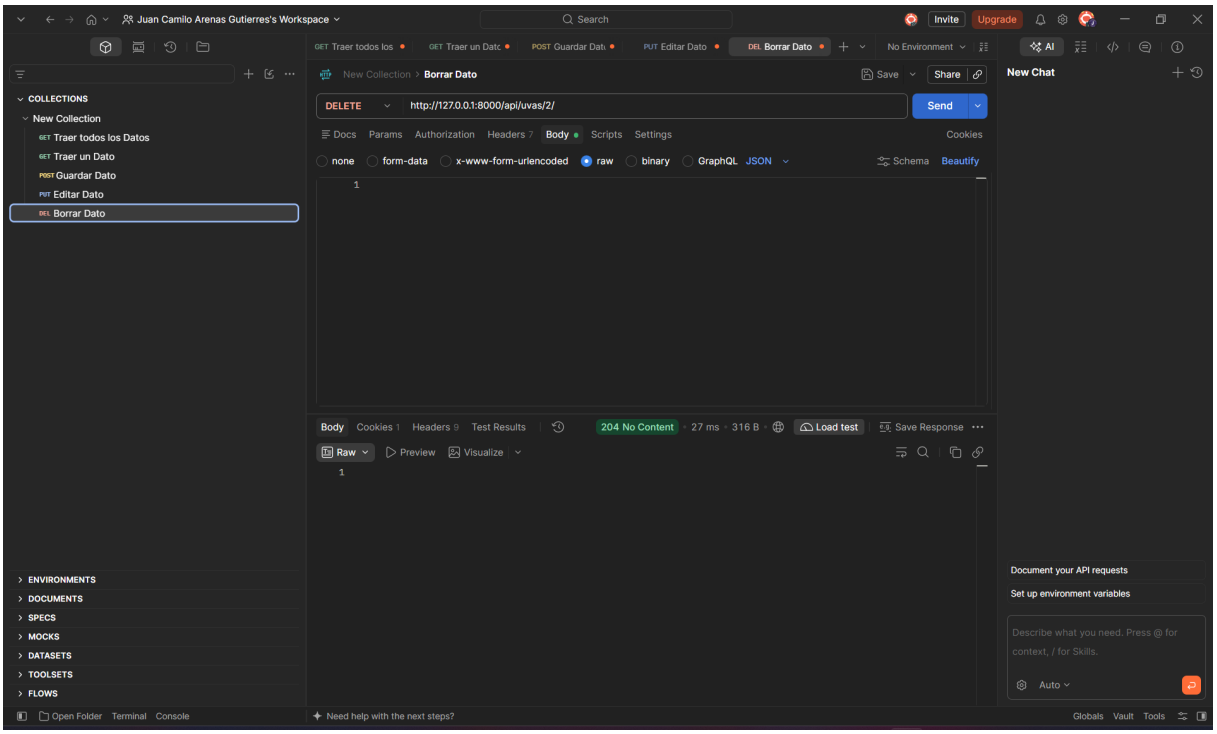


Figura 6. Eliminación del registro con respuesta 204 No Content.

4. Consola / Terminal de Django

El servidor se ejecutó dentro del entorno virtual (`.venv`) con el comando `python manage.py runserver`. La terminal registra cada petición recibida con la fecha, el método, la ruta, el código de estado y el tamaño de la respuesta en bytes. El registro coincide con las pruebas realizadas en Postman.

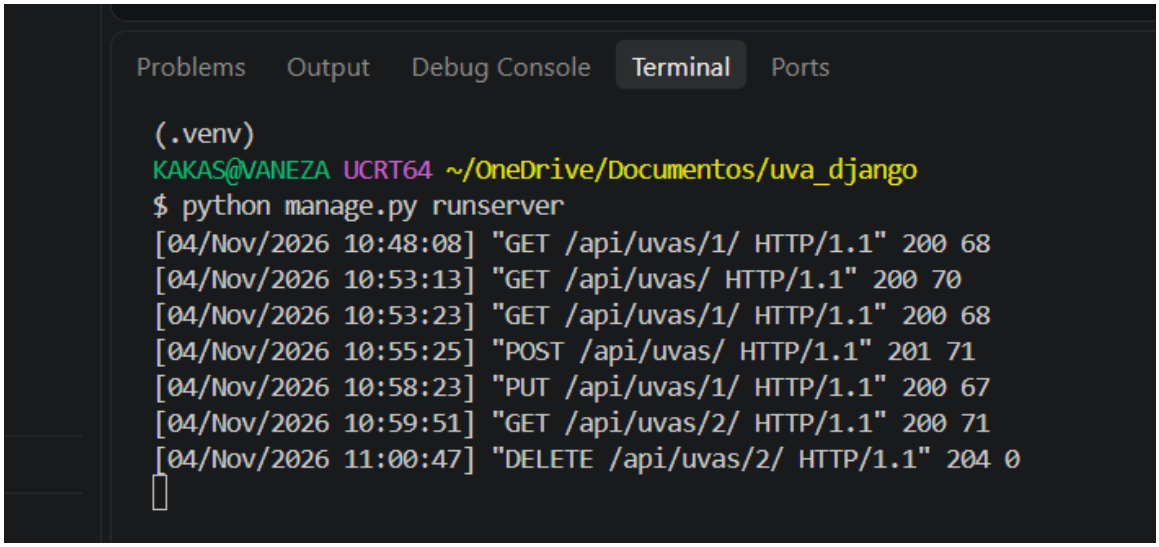


Figura 7. Registro de peticiones HTTP en el servidor de desarrollo.

Hora	Petición	Código	Corresponde a
10:48:08	GET /api/uvas/1/	200	Prueba inicial de consulta por ID
10:53:13	GET /api/uvas/	200	Listado de registros (3.1)
10:53:23	GET /api/uvas/1/	200	Consulta por ID (3.2)
10:55:25	POST /api/uvas/	201	Creación de registro (3.3)
10:58:23	PUT /api/uvas/1/	200	Edición de registro (3.4)
10:59:51	GET /api/uvas/2/	200	Verificación del registro creado
11:00:47	DELETE /api/uvas/2/	204	Eliminación de registro (3.5)

5. Conclusiones

- La API REST implementa correctamente las operaciones CRUD sobre el recurso uvas.
- Cada operación devolvió el código HTTP esperado: 200 (GET y PUT), 201 (POST) y 204 (DELETE).
- Los registros de la terminal coinciden con las peticiones enviadas desde Postman, lo que confirma que el servidor las procesó sin errores.
- La edición y la eliminación se reflejaron en los datos, lo que demuestra la persistencia en la base de datos.